

ROS2 입문 1주차 요약본

1. ROS2 프로그래밍 기초

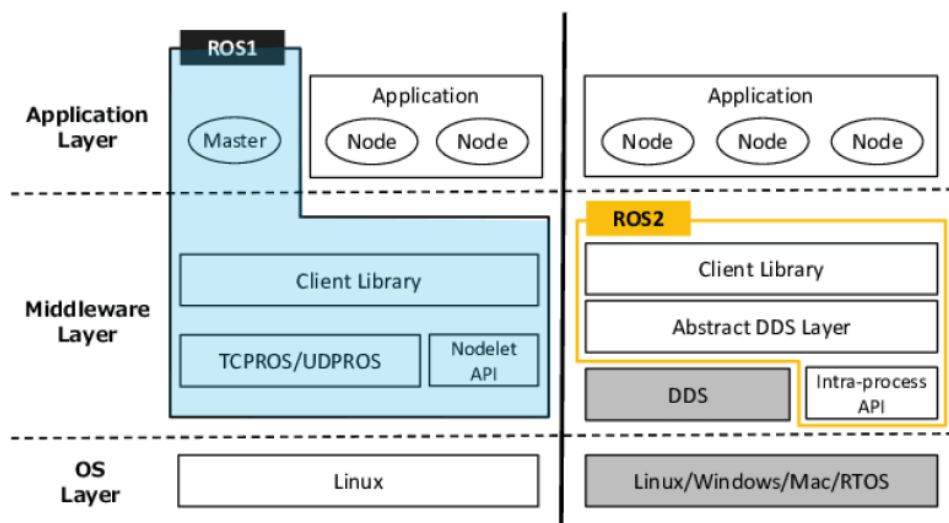
1.1 ROS2 프로그래밍이란?

- Message
 - 토픽(Topic)
 - msg 파일
 - 비동기식, 단방향
 - 지속적인 센서 데이터
 - 서비스(Service)
 - srv 파일
 - 동기식, 요청 ↔ 응답
 - 짧은 명령 요청
 - 액션(Action)
 - action 파일
 - 혼합형, Goal ↔ Feedback ↔ Result
 - 장시간 작업 명령

1.2 ROS1 vs ROS2 비교

항목	ROS1	ROS2
운영 체제 지원 (OS Layer)	Linux만 지원	Linux, Windows, macOS, RTOS 등 멀티 플랫폼 지원
미들웨어 계층 (Middleware Layer)	자체 프로토콜 (TCPROS / UDPROS) Nodelet API	DDS(Data Distribution Service) 기반 추상 DDS 계층 + Intra-process 통신 API
중앙 관리 시스템	Master 노드 필수 (중앙 집중형)	Master 노드 없음 (분산형, discovery 자동화)

노드 통신 방식	Master가 Publisher-Subscriber 간 통신을 중재	DDS가 자동 discovery 및 직접 통신 수행
노드 구성	Nodelet 사용 시 복잡도 높음	Intra-process 통신으로 한 프로세스 내 노드 간 효율적 메시지 전달
클라이언트 라이브러리	<code>roscpp</code> , <code>rospy</code> 등 단일 구조	<code>rclcpp</code> , <code>rclpy</code> , <code>rcl</code> 등 추상화 계층 제공
확장성 및 실시간성	제한적	DDS 기반으로 확장성, 실시간성, 보안성 우수
멀티 노드 실행 구조	노드 간 통신 설정이 수동	자동 discovery + QoS 기반 유연한 구성 가능



1.3 ROS2에서의 프로그래밍 역할

- 노드 개발
 - ROS2는 로봇 시스템을 여러 독립적인 노드로 나누어 구성하며, 각 노드는 특정 기능을 수행함
 - 프로그래밍을 통해 노드를 개발하고, 다른 노드와 데이터를 교환하거나 상호작용하게 할 수 있음
- 토픽(Topic)과 메시지(Message) 관리
 - ROS2에서는 노드 간 통신이 토픽을 통해 이루어지며, 각 노드는 특정 토픽을 발행하거나 구독하여 데이터를 주고받음

- 프로그래밍을 통해 메시지 형식을 정의하고, 이를 통해 로봇 시스템의 구성 요소들이 정보를 효율적으로 교환 할 수 있도록 함
- 서비스(Service)와 액션(Action)
 - ROS2는 요청-응답 방식의 서비스를 통해 노드 간 통신을 구현할 수 있으며, 액션을 이용해 시간이 오래 걸리는 작업도 비동기적으로 처리할 수 있음
- 로봇 하드웨어 제어
 - ROS2는 센서, 모터, 로봇 팔 등 로봇 하드웨어 제어를 위한 다양한 라이브러리와 드라이버를 제공하며, 프로그래밍을 통해 하드웨어 제어 알고리즘과 데이터 처리 코드를 구현할 수 있음
- 로봇 동작의 논리 구현
 - ROS2는 로봇의 복잡한 동작을 제어할 수 있는 도구를 제공하며, 이를 통해 경로 계획이나 장애물 회피 등 로봇의 동작 논리를 프로그래밍 할 수 있음
- 시뮬레이션(Simulation)
 - ROS2는 Gazebo 등의 시뮬레이터와 연동하여, 실제 하드웨어 없이 로봇 동작을 테스트하고 성능을 평가 할 수 있음
- 분산 시스템 구현
 - ROS2는 분산 시스템 기반으로 설계되어, 여러 컴퓨터에서 실행되는 노드 간 통신을 통해 작업을 분담하고 로봇 시스템을 조정할 수 있음

1.4 프로그래밍 규칙

- Python Style
 - 공백(Space) 4칸 사용
 - 탭(Tab) 문자 금지
 - 자료형에 따라 적절한 괄호 사용
 - ex) 리스트: `[]` , 딕셔너리: `{}` , 튜플: `()`
 - 코드 스타일 검사 도구 : `ament_flake8` 사용
 - 문자열은 큰따옴표(") 대신 작은따옴표(') 사용 권장

1.5 ROS2 Tips

- Underlay vs Overlay

항목	Underlay	Overlay
정의	기본 ROS2 설치 환경 또는 하위 워크스페이스	기존 underlay 위에 덮어쓰는 상위 워크스페이스
위치 예시	<code>/opt/ros/humble</code> (ROS2 기본 설치 위치)	<code>~/ros2_ws</code> , <code>~/dev_ws</code> 등 사용자가 만든 워크스페이스
목적	ROS2 전체 시스템 또는 하위 기반 제공	새로운 패키지를 추가하거나 수정할 때 사용
설정 방법	<code>source /opt/ros/humble/setup.bash</code>	<code>source install/setup.bash</code>
종첩 사용	다른 워크스페이스의 기반이 됨	underlay를 기반으로 동작함
특징	수정 불가(대개 시스템 설치)	자유롭게 수정, 빌드, 테스트 가능
활용 예시	ROS2 전체 기능 사용을 위한 기본 환경 구성	사용자 정의 패키지 개발 및 테스트

- Setup.bash

- `/opt/ros/humble` 에 설치된 ROS의 릴리스 환경을 설정하는데 사용

- local_setup.bash

- 현재 작업 중인 ROS의 워크스페이스의 환경 설정에 사용
- 상위 작업 공간에 대한 설정은 미포함

- colcon_cd

```
# 셸의 현재 작업 디렉터리를 패키지 디렉터리로 빠르게 변경
$ colcon_cd 패키지이름
$ source /usr/share/colcon_cd/function/colcon_cd.sh

# 기본 작업 폴더를 ~/robot_ws
$ export _colcon_cd_root=~ /robot_ws
```

- ROS_DOMAIN_ID

DDS(Data Distribution Service) 도메인

- 네트워크상의 노드들이 서로를 찾고 통신 할 수 있도록 함
- ROS2는 DDS라는 미들웨어를 사용하여 노드 간의 메시지 전달과 통신을 처리

(0 ~ 232) ID값을 선택 가능

```
$ export ROS_DOMAIN_ID=8
```

```
$ ros2 run demo_nodes_cpp talker
```

```
[INFO]: Publishing:'Hello World: 1'
```

```
[INFO]: Publishing:'Hello World: 2'
```

```
$ export ROS_DOMAIN_ID=8
```

```
[INFO]: I heard: [Hello World: 12]
```

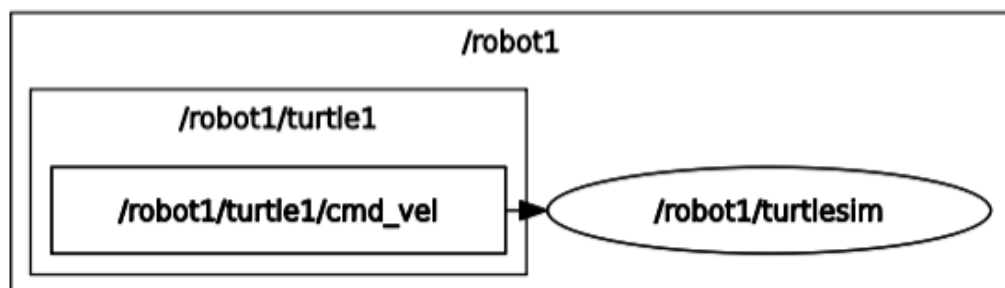
```
[INFO]: I heard: [Hello World: 13]
```

- Namespace

namespace robot1 변경

```
$ ros2 run turtlesim turtlesim_node --ros-args -r __ns:=/robot1
```

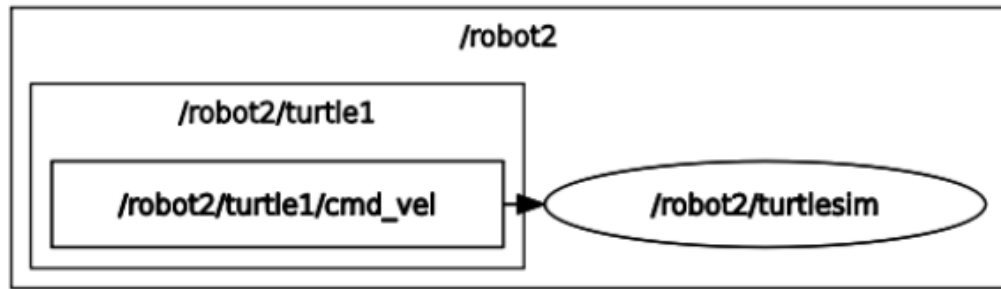
```
$ rqt_graph
```



namespace robot1 변경

```
$ ros2 run turtlesim turtlesim_node --ros-args -r __ns:=/robot2
```

```
$ rqt_graph
```



- Uninstall

```

# ROS2 관련 패키지 삭제
$ sudo apt remove ~nros-humble-* && sudo apt autoremove
# ROS2 repository 삭제
$ sudo rm /etc/apt/sources.list.d/ros2.list

```

1.5 패키지

- package.xml
 - 메타 정보 포함 (신분증 역할)
 - 패키지 의존성 설치 시 rosdep이 이 파일의 정보를 기반으로 설치

```

ros2_ws/
├── src/
│   ├── my_pkg_1/
│   │   └── package.xml ← 여기에 의존성 정보 있음
│   └── my_pkg_2/
│       └── package.xml

```

- setup.cfg

```

# setup.cfg
[metadata]
name = my_python_pkg
version = 0.1.0
description = 예시 Python 패키지

```

```
author = Your Name
license = MIT
```

```
[options]
packages = find:
install_requires =
    rclpy
    std_msgs
```

- setup.py

```
# setup.py
from setuptools import setup

package_name = 'my_python_pkg' # 패키지 이름 (폴더명과 동일해야 함)

setup(
    name=package_name,
    version='0.1.0',
```

- ROS2에서의 역할

- Python 기반의 ROS2 패키지에 대해 colcon은 setup.cfg(및 setup.py)를 사용하여 패키지의 설치를 처리
- setup.cfg, setup.py는 setuptools를 통해 패키지를 빌드하고 설치하는 방법에 대한 구성 정보를 제공

항목	<u>setup.py</u>	setup.cfg
접근 방식	프로그래밍 방식으로 패키지 설정을 제공	선언적 방식으로 설정을 제공
유연성	동적 계산과 사용자 정의 명령어를 지원하는 높은 유연성을 제공	보다 간단하고 명확한 패키지 설정을 지향
사용 추세	패키지 의존성을 해결해주지 않으므로 가능한 한 한 <u>setup.py</u> 파일은 최소화하거나 제거	현대의 Python 패키징은 setup.cfg를 통한 선언적 설정을 선호

2. 토픽, 서비스 패키지

2.1 Topic 패키지 생성 (py_pubsub)

```
# ros2_ws/src 디렉토리 생성
mkdir -p ros2_ws/src
cd ros2_ws/src
# py_pubsub package 생성
ros2 pkg create --build-type ament_python py_pubsub
# 생성된 Package 확인
ls
# Tree를 이용한 내부 구조 확인
tree
```

- Python으로 publisher와 subscriber node를 생성하고 실행하기
→ Topic상으로 message를 전송/수신하는 역할 (talker/listener)
- Package.xml 설정
 - 의존성(dependencies) 추가

```
<?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>py_pubsub</name>
  <version>0.0.0</version>
  <description>Examples of minimal publisher/subscriber using rclpy</description>
  <maintainer email="you@email.com">i</maintainer>
  <license>Apache License 2.0</license>
```

```
# 해당 ROS 패키지가 실행되기 위해 필요한 의존성을 지정하는데 사용
<exec_depend>rclpy</exec_depend> # rclpy: ROS2의 python 클라이언트 라이브러리
<exec_depend>std_msgs</exec_depend> # std_msgs: ROS에서 기본적으로 제공하는 메시지 타입들의 모음
```

```
# 패키지 개발 시 테스트 자동화를 위한 환경 구성에 필수적
<test_depend>ament_copyright</test_depend>
<test_depend>ament_flake8</test_depend>
<test_depend>ament_pep257</test_depend>
```



```

<test_depend>python3-pytest</test_depend>
<export>
  <build_type>ament_python</build_type>
</export>
</package>

```

2.2 Service 패키지 생성

```

# 패키지 생성
ros2 pkg create --build-type ament_python py_srvcli --dependencies rclpy
example_interfaces
# 패키지 빌드
colcon build --packages-select py_srvcli
# 노드 실행 (터미널1)
ros2 run py_srvcli service
# (터미널2)에서 실행
ros2 run py_srvcli client 2 3

```

3. 액션 패키지

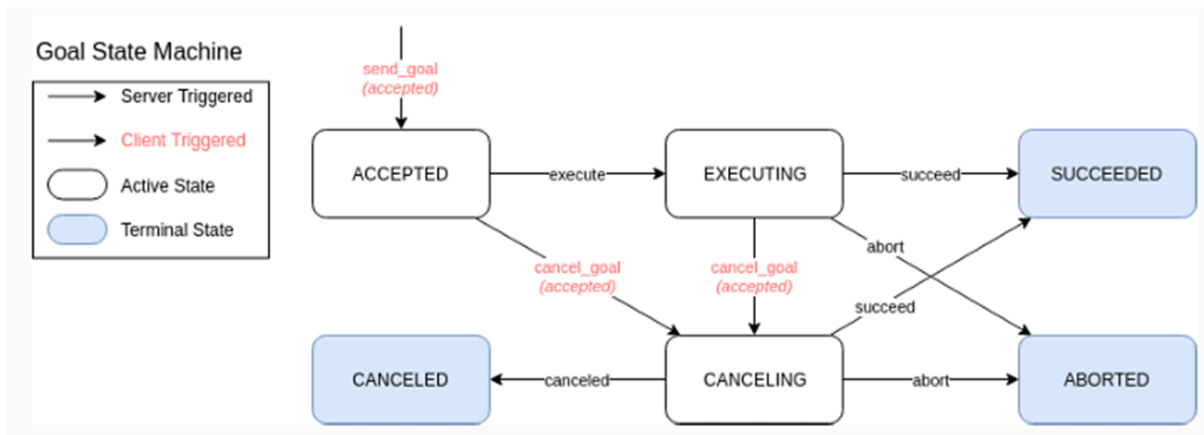
3.1 Action 패키지 생성

- Action 동작 다이어그램



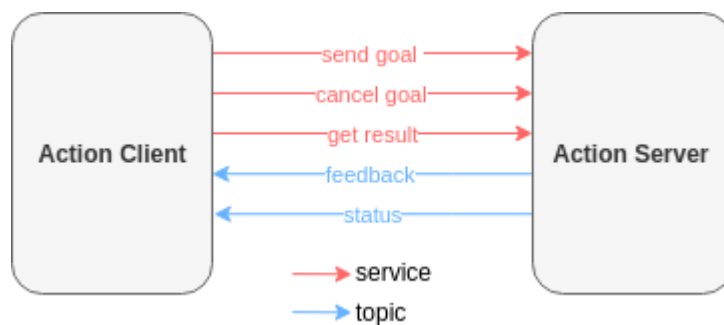
- **Goal State Machine**

- 액션 서버는 클라이언트로부터 수신한 각 목표에 대해 상태 머신을 유지, 거부된 목표는 상태 머신에 포함되지 않음
- 활성 상태
 - **ACCEPTED** : 목표가 수락되었으며 실행을 대기
 - **EXECUTING** : 목표는 현재 액션 서버에서 실행
 - **CANCELING** : 클라이언트가 목표 취소를 요청했고 액션 서버가 취소 요청을 수락
- 최종 상태
 - **SUCCEEDED** : 액션 서버가 목표를 성공적으로 달성
 - **ABORTED** : 목표가 외부 요청 없이 작업 서버에 의해 종료
 - **CANCELED** : 액션 클라이언트의 외부 요청 이후 목표가 취소
- 설계된 동작에 따라 액션 서버에서 트리거되는 상태 전환
 - **execute** : 승인된 목표의 실행을 시작
 - **succeed** : 목표가 성공적으로 완료 알림
 - **abort** : 목표 처리 중에 오류가 발생하여 중단 알림
 - **canceled** : 목표 취소가 성공적으로 완료 알림
- 액션 클라이언트에 의해 트리거되는 상태 전환 알림
 - **send_goal** : 목표가 액션 서버로 전송, 상태 머신은 액션 서버가 목표를 수락할 때만 시작
 - **cancel_goal** : 액션 서버에 목표 처리를 중단하도록 요청, 액션 서버가 목표 취소 요청을 수락한 경우에만 전환이 발생



3.2 Action 인터페이스 정의

- Action 이란?
- ROS 2의 통신 개념 중 하나로, 장시간 소요되는 비동기 작업을 수행하고, 진행 중 피드백과 취소 기능을 지원하는 구조



- **Action Server**
 - 하나 이상의 액션 클라이언트로부터 목표 수락 또는 거부
 - 목표가 수신되고 수락되면 작업을 실행
 - 완료된 작업의 결과(성공, 실패 또는 취소 여부 포함)를 결과를 요청한 클라이언트에게 전송
- **Action Client**
 - 액션 서버로 목표 전송
- **Goal (목표)**
 - 액션이 무엇을 달성해야 하는지, 그리고 어떻게 달성해야 하는지를 설명
 - 액션 실행 요청을 받으면 액션 서버로 전송

- **Result (결과)**

- 작업의 결과를 설명
- 작업 실행이 성공적으로 종료되었는지 여부와 관계없이 서버에서 클라이언트로 전송

- **Feedback (피드백)**

- 작업 완료까지의 진행 상황을 나타냄
- 작업 실행 시작부터 완료 전까지 작업 서버에서 해당 작업의 클라이언트로 전송
- 클라이언트는 이 데이터를 사용하여 작업 실행 진행 상황을 파악

- Action 동작 흐름

1. **Goal 전송:** 클라이언트가 서버에 작업 요청.
2. **Goal 수락:** 서버가 요청 수락/거절.
3. **실행 및 피드백:** 서버가 작업 실행하며 클라이언트에 피드백 전달.
4. **결과 전송:** 작업 완료 시 성공/실패/취소 결과 전송.
5. **취소 처리:** 클라이언트가 요청 시 서버는 Goal 취소 처리 가능.

3.3 Action 패키지 생성

- ros2 action 명령어

명령어	설명
<code>ros2 action list</code>	액션 서버/클라이언트 목록
<code>ros2 action info</code>	액션 타입 조회
<code>ros2 action show</code>	액션 메시지 구조 확인
<code>ros2 action send_goal</code>	액션 실행 및 피드백/결과 확인
<code>ros2 action status</code>	Goal 상태 모니터링
<code>ros2 action feedback</code>	실시간 피드백 출력

- .action 파일

```
# Fibonacci.action

# Goal
int32 order

---
# Result
int32[] sequence

---
# Feedback
int32[] partial_sequence
```

4. 인터페이스 패키지

4.1 인터페이스

- ROS에서 노드 사이에 데이터를 전송 시 사용되는 토픽(Topic), 서비스(Service), 액션(Action)에서 사용되는 데이터 타입
- 토픽은 msg파일, 서비스는 srv파일, 액션은 action 파일에 인터페이스가 정의

종류	설명	예시
msg (message)	Topic 통신에서 사용되는 데이터 형식	std_msgs/msg/String, sensor_msgs/msg/LaserScan
srv (service)	요청/응답 통신용 데이터 형식	example_interfaces/srv/AddTwoInts
action	장시간 작업 요청용 데이터 형식 (goal, feedback, result)	nav2_msgs/action/NavigateToPose

- **.msg** : Topic 통신에 사용되는 데이터 구조, 단방향 통신 (요청 없이 데이터 전송)

```
# 메시지 구조
# .msg
<필드타입> <필드이름>
<필드타입> <필드이름>
```

<필드타입> <필드이름>

```
# 예시) std_msgs/msg/String.msg
string data
```

- **.srv** : 요청/응답 방식 통신, ---구분자로 섹션을 구분

```
# 메시지 구조
# REQUEST
<필드타입> <필드이름>
---
# RESPONSE
<필드타입> <필드이름>

# 예시) AddTwoInts.srv
int64 a
int64 b
---
int64 sum
```

- **.action** : 장시간 실행되는 작업을 위한 통신

```
# 메시지 구조
# Goal
<필드타입> <목표필드>
---
# Result
<필드타입> <결과필드>
---
# Feedback
<필드타입> <피드백필드>

# 예시) Fibonacci.action
int32 order
---
int32[] sequence
```

```
---
int32[] sequence
```

- 기본값과 상수값

항목	상수 (Constant)	기본값 (Default value)
선언 형식	타입 NAME=값	타입 필드이름 기본값
변경 가능	불가 (컴파일타임 고정)	가능 (런타임 수정 가능)
사용 위치	msg, srv, action 모두	msg만 완전 지원
사용 목적	코드 내 고정값 표현	필드 값 초기화 용도
예시	int8 OK= 1	string name "Robot"

4.2 인터페이스 패키지 생성 (my_robot_interfaces)

```
# 패키지 생성
cd ~/ros2_ws/src/
ros2 pkg create --build-type ament_cmake my_robot_interfaces

# 폴더 생성
# 파일명은 반드시 카멜케이스(CamelCase)만 사용, 만약 첫 문자가 소문자일 경우 빌드 시 오류 발생
msg/Status.msg
srv/AddTwoInts.srv
action/MyAction.action
action/Navigate.action

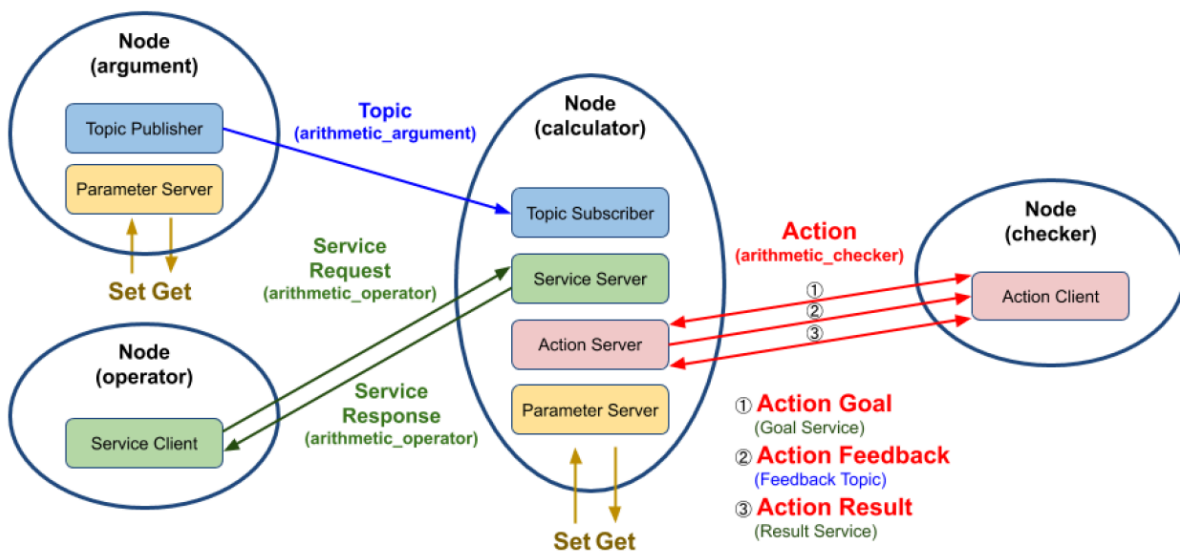
# Package.xml 파일 수정
# CMakeLists.txt 파일 수정
# 빌드
cd ../..
colcon build --packages-select ros_study_msgs
```

4.3 인터페이스 패키지 생성 (ex_calculator)

패키지 설계

- argument node
 - arithmetic_argument 토픽으로 현재 시간과 변수 a와 b를 퍼블리시함
- operator node
 - arithmetic_operator 서비스를 통해 calculator 노드에게 연산자(+,-,*,/)를 서비스 요청값으로 보냄
- calculator node
 - **calculator** 노드는 토픽으로 받은 변수 a, b와 서비스 요청으로 받은 연산자를 이용해 계산 후 결과를 응답하고, 액션을 통해 누적된 결과값과 연산식을 피드백
 - 누적 결과가 액션 목표값을 초과하면, 최종 결과와 연산식을 액션 결과로 반환함
- checker node
 - 연산 결과값의 누적 한계치를 액션 목표값으로 전달

<패키지 구성>



- 의존 패키지 설치

```
$ rosdep install --from-paths src --ignore-src -r -y
```

```
# CMakeLists.txt 파일 수정
set(msg_files
```



```

    "msg/MyMsg.msg"
    "msg/ArithmeticArgument.msg")
set(srv_files
    "srv/MySrv.srv"
    "srv/ArithmeticOperator.srv")
set(action_files
    "action/MyAction.action"
    "action/ArithmeticChecker.action")

```

생성한 세개의 폴더 및 파일 작성

- msg/ArithmeticArgument.msg
 - # Messages
 - builtin_interfaces/Time stamp
 - float32 argument_a
 - float32 argument_b
- srv/ArithmeticOperator.srv
 - # Constants
 - int8 PLUS = 1
 - int8 MINUS = 2
 - int8 MULTIPLY = 3
 - int8 DIVISION = 4
 - # Request
 - int8 arithmetic_operator
 - # Response
 - float32 arithmetic_result
- action/ArithmeticChecker.action
 - # Goal
 - float32 goal_sum
 - # Result
 - string[] all_formula
 - float32 total_sum
 - # Feedback
 - string[] formula

```

param
└── arithmetic_config.yaml

```

/**: # namespace and node name

```
ros__parameters:
  qos_depth: 30
  min_random_num: 0
  max_random_num: 9

# 빌드
$ colcon build --packages-select ros_study_msgs_ex_calculator
# 실행
$ ros2 run ex_calculator calculator
$ ros2 run ex_calculator argument
$ ros2 run ex_calculator operator
$ ros2 run ex_calculator checker
```

5. rclpy 이해

5.1 rclpy란?

- 정의
 - ROS 2의 Python 클라이언트 라이브러리로, Python에서 노드를 생성하고 Topic, Service, Action 등 ROS 2 통신 기능을 사용할 수 있게 함
- 기반 구조
 - C++기반 ROS 2 core 라이브러리인 **rc1** 을 Python으로 래핑한 것이 특징
- 구성요소
 - 노드(Node)
 - ROS2 시스템의 기본 실행 단위, pub/sub, service/client 등을 통한 통신 담당
 - 퍼블리셔와 서브스크라이버
 - 퍼블리셔 → 토픽 → 서브스크라이버, rclpy에서는 간단한 API로 pub/sub를 설정
 - 서비스와 클라이언트
 - 서비스는 요청-응답 방식의 통신을 처리하며, 클라이언트는 특정 요청을 보내고 응답 대기

- 액션
 - 긴 시간 동안 실행되는 작업을 요청하고 그 결과를 받을 수 있는 기능 ex) Motion Planning
- 파라미터 서버
 - ROS2 시스템에서 전역적 혹은 로컬 파라미터를 설정하고 가져올 수 있는 시스템

5.2 Multi Thread

멀티스레드 지원

- rclpy는 멀티스레드 실행 지원 가능
- 서비스와 퍼블리셔가 동일한 노드 내에서 동시에 작동하는 경우, MultiThreaded Executor를 사용 가능

최적화 팁

- 토픽 Qos 설정
 - 네트워크 환경이나 메시지의 중요도에 따라 Qos설정을 맞춰야 함
 - 메시지 유실이 치명적이지 않은 센서 데이터의 경우 Best Effort 방식을 사용할 수 있음
- 파라미터 최적화
 - 파라미터 서버를 활용해, 노드의 설정 값을 유연하게 바꾸면서 성능 튜닝 가능

```
from rclpy.executors import MultiThreadedExecutor
executor = MultiThreadedExecutor()
executor.add_node(node)
executor.spin()
```

executor란?

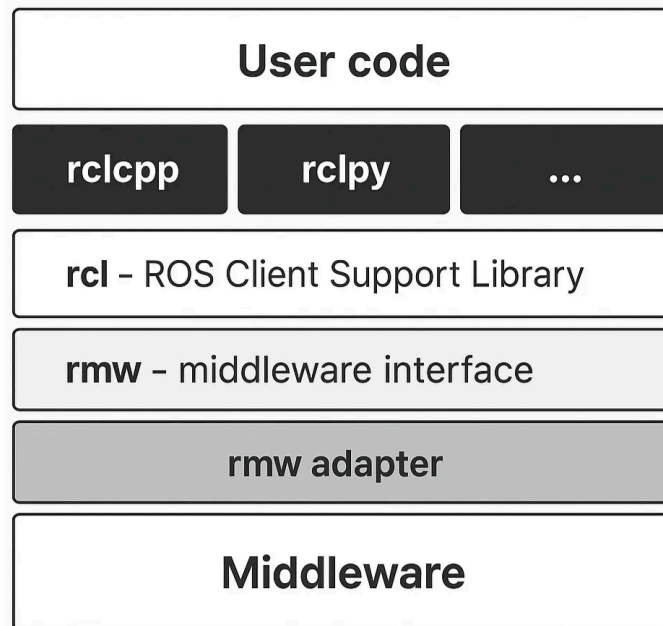
- ROS2에서 콜백을 실행하는 구성 요소
- 메시지 수신, 서비스 요청 처리, 타이머 이벤트 등 다양한 이벤트에 대한 응답 실행
- 시스템의 비동기 처리와 효율성 보장
- 노드가 구독하는 데이터나 이벤트를 적절한 콜백으로 처리하여 메시지의 QoS를 지원
- SingleThreadExecutor: 단일 스레드에서 콜백을 순차적으로 처리
- StaticSingleThreadedExecutor : 단일 스레드에서 정적으로 콜백을 실행

- MultiThreadExecutor: 여러 스레드에서 콜백을 병렬적으로 처리



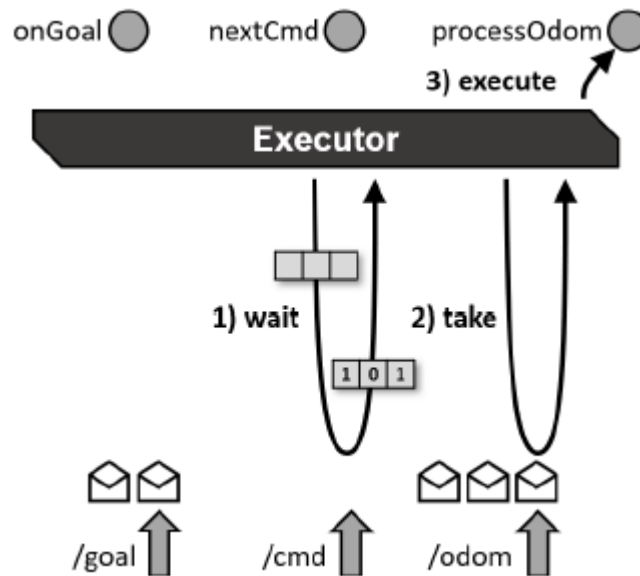
5.3 ROS2 계층구조

- User code
 - 사용자가 작성한 코드
- rcl
 - 클라이언트 라이브러리로, 사용자 코드와 미들웨어를 연결
- rmw
 - 미들웨어와 클라이언트 간의 인터페이스 역할
- rmw adapter
 - rmw와 미들웨어를 연결하는 중간 계층
 - ROS2는 특정 DDS 구현체에 종속되지 않고, 다양한 미들웨어를 지원할 수 있는 유연성을 가짐
- Middleware
 - 실제 메시지 전달과 QoS 설정을 처리하는 계층
 - 애플리케이션 또는 구성 요소 간에 하나 이상의 종류의 통신 또는 연결을 가능하게 하는 소프트웨어



5.4 Executor 동작 과정

- wait
 - 미들웨어에 메시지가 도착할 때까지 대기
- take
 - 새로운 메시지 도착 시, 해당 메시지를 가져옴
 - 위 과정에서 미들웨어에 저장된 메시지가 클라이언트로 전달됨
- execute
 - 메시지 처리를 위한 콜백함수(onGoal, nextCmd, processOdom) 실행
- 동작 순서
 - 이벤트 감지 → 콜백 큐 생성(Queue에 수집) → 콜백 실행



	Executor	spin()
역할	콜백 관리 및 실행	이벤트 루프 실행
설명	- 노드에 포함된 콜백(토픽, 서비스, 타이머 등)을 관리하고, 이벤트가 발생하면 적절한 콜백을 실행하는 역할 수행
 - <code>SingleThreadedExecutor</code>, <code>MultiThreadedExecutor</code> 등 다양한 종류가 있으며, 콜백 실행 방식에 따라 동작 방식이 다름	- <code>Executor</code> 가 이벤트를 처리하는 루프를 실행
 - <code>spin()</code> 이 호출되면 노드는 콜백이 발생하기를 기다리며, 이벤트가 감지될 때 콜백을 실행
 - <code>Executor</code> 가 동작할 수 있는 환경을 유지하는 루프이며, 명시적으로 종료하지 않으면 프로그램이 종료될 때까지 계속 실행